SOLICITATION ACADEMY

SAAS

Complete Handoff Guide

Version 1.0 — Current Build State

STACK

Next.js 16 • TypeScript • Supabase

Tailwind v4 • Claude AI

PREPARED FOR VENCHUR INDUSTRIAL SUPPLY

Table of Contents

Right-click the TOC below and select "Update Field" to refresh page numbers after opening.

1. Project Overview

Solicitation Academy is a subscription SaaS web application teaching small business contractors how to sell physical products to the Defense Logistics Agency through the DIBBS platform.

Two Core Features

Curriculum Mode — 30 structured lessons across 4 progressive levels (101, 201, 301, 401) with an always-on AI Tutor per lesson.

Live PDF Analysis — Students upload real DLA solicitation PDFs and receive a 14-tab AI-generated breakdown with context-aware Analysis Tutor.

Design System

Warm editorial aesthetic — ivory backgrounds (#F6F1E8), amber primary (#B26B1F), Newsreader serif for headlines, Inter sans for body. Level colors: amber 101 / sage 201 / navy 301 / rust 401.

2. Current Build Status

Completed Phases

Phase	Name	Status
1	Foundation setup	Complete
2	Authentication	Complete
3	Database schema	Complete
4	Dashboard & navigation	Complete
5	Curriculum system (30 lessons)	Complete
6	AI Tutor for lessons	Complete
7	PDF upload & analysis	Complete

Remaining Phases

Phase	Name	Status
8	Stripe subscriptions (4-day trial)	Not started

Phase	Name	Status
9	Admin dashboard	Not started
10	Polish & error handling	Not started
11	Deployment to Vercel	Not started
12	Final documentation	Not started

3. Tech Stack (Current Versions)

Category	Tool / Version
Framework	Next.js 16.2.4 (App Router + Turbopack)
Language	TypeScript
UI Library	React 19
Styling	Tailwind CSS v4 (CSS-first config)
Components	shadcn/ui (Radix primitives)
Database	Supabase (PostgreSQL + Auth + Storage)
AI	Anthropic Claude Sonnet 4.5
PDF Parsing	pdf-parse (server-side)
Icons	Lucide React
Fonts	Newsreader, Inter, JetBrains Mono
Package Manager	npm
Node	v24.15.0

Required Node Environment Variables

Create **.env.local** in the project root:

COPY • .ENV.LOCAL

```
NEXT_PUBLIC_SUPABASE_URL=your_supabase_project_url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_supabase_anon_key
```

```
ANTHROPIC_API_KEY=your_anthropic_api_key
```

4. Complete File Structure

Every file and folder in the project, annotated:

COPY • FILE TREE

```
solicitation-academy-saas/
│
├── app/                                    # Next.js App Router
│   ├── (auth)/                             # Route group for auth
│   │   ├── login/
│   │   │   ├── page.tsx                    # Login page (2-column editorial)
│   │   │   └── login-form.tsx              # Client form component
│   │   └── signup/
│   │       ├── page.tsx                    # Signup page
│   │       └── signup-form.tsx             # Client form component
│   │
│   ├── (dashboard)/                        # Route group for authenticated
│   │   ├── layout.tsx                      # Shell: sidebar + topbar
│   │   │
│   │   ├── dashboard/
│   │   │   └── page.tsx                    # Main dashboard with hero
│   │   │
│   │   ├── lesson/
│   │   │   └── [moduleId]/
│   │   │       ├── page.tsx                # Lesson page (server)
│   │   │       ├── lesson-view.tsx         # Layout: article + tutor
│   │   │       ├── lesson-content.tsx      # HTML renderer
│   │   │       └── tutor-panel.tsx         # AI Tutor sidebar
│   │   │
│   │   ├── upload/
│   │   │   ├── page.tsx                    # Upload page
│   │   │   └── upload-form.tsx             # PDF upload with progress
│   │   │
│   │   └── analysis/
│   │       └── [sessionId]/
│   │           ├── page.tsx                # Results page (server)
│   │           ├── analysis-view.tsx       # Tab navigation + layout
│   │           ├── analysis-tutor.tsx      # Context-aware tutor
│   │           └── tabs/
│   │               ├── overview-tab.tsx    # Overview tab
│   │               ├── section-tab.tsx     # Sections A-J tab
│   │               ├── history-tab.tsx     # Price history tab
│   │               ├── checklist-tab.tsx   # Compliance tab
│   │               └── gonogo-tab.tsx      # Go/No-Go tab
│   │
│   ├── auth/
│   │   └── callback/
```

```
│   │       └── route.ts                    # Email verification callback
│   │
│   ├── api/
│   │   ├── tutor/
│   │   │   └── route.ts                    # Lesson AI Tutor endpoint
│   │   ├── analysis-tutor/
│   │   │   └── route.ts                    # Analysis AI Tutor endpoint
│   │   └── upload-solicitation/
│   │       └── route.ts                    # PDF analysis pipeline
│   │
│   ├── globals.css                         # Tailwind v4 + design tokens
│   ├── layout.tsx                          # Root layout with fonts
│   └── page.tsx                            # Home (redirects based on auth)
│
├── components/
│   ├── layout/
│   │   ├── sidebar.tsx                     # Left nav with levels
│   │   ├── top-bar.tsx                     # Top with breadcrumb, pills
│   │   ├── theme-toggle.tsx                # Dark/light switcher
│   │   └── user-menu.tsx                   # Avatar dropdown
│   │
│   ├── ui/                                 # shadcn/ui components
│   │   ├── avatar.tsx
│   │   ├── badge.tsx
│   │   ├── button.tsx
│   │   ├── card.tsx
│   │   ├── dialog.tsx
│   │   ├── dropdown-menu.tsx
│   │   ├── form.tsx
│   │   ├── input.tsx
│   │   ├── label.tsx
│   │   ├── separator.tsx
│   │   ├── skeleton.tsx
│   │   ├── sonner.tsx
│   │   ├── tabs.tsx
│   │   └── textarea.tsx
│   │
│   ├── progress-ring.tsx                   # SVG progress circle
│   └── theme-provider.tsx                  # Theme context
│
├── lib/
│   ├── supabase/
│   │   ├── client.ts                       # Browser Supabase client
│   │   ├── server.ts                       # Server Supabase client
│   │   └── middleware.ts                   # Auth middleware helper
│   │
│   ├── curriculum.ts                       # 30 modules + level structure
│   ├── lessons.ts                          # All 30 lesson HTML content
│   ├── analysis-engine.ts                  # Types + Claude analysis prompt
│   ├── packaging-codes.ts                  # MIL-STD-2073-1E code database
│   └── utils.ts                            # cn() helper from shadcn
│
├── types/
```

```
│   └── database.types.ts                  # Auto-generated from Supabase
│
├── public/                                # Static assets
│
├── .env.local                             # Secret keys (not committed)
├── .gitignore
├── components.json                        # shadcn config
├── next.config.ts
├── package.json
├── postcss.config.mjs
├── proxy.ts                               # Next.js 16 middleware
├── tsconfig.json
└── README.md
```

5. Database Schema (Supabase)

Tables

profiles

Extended user profile linked to auth.users

COPY • SQL

```
id                      UUID PRIMARY KEY (refs auth.users)
email                   TEXT NOT NULL
full_name               TEXT
avatar_url              TEXT
stripe_customer_id      TEXT UNIQUE
subscription_status     TEXT (trial/active/canceled/expired/past_due)
subscription_id         TEXT
trial_ends_at           TIMESTAMPTZ
current_period_end      TIMESTAMPTZ
theme                   TEXT DEFAULT 'light'
font_scale              INTEGER DEFAULT 100
current_module          TEXT DEFAULT '101_1'
current_level           TEXT DEFAULT '101'
total_time_mins         INTEGER DEFAULT 0
is_admin                BOOLEAN DEFAULT FALSE
created_at, updated_at, last_active
```

module_progress

Per-user lesson tracking

COPY • SQL

```
id                TEXT PRIMARY KEY
user_id           UUID (refs auth.users)
module_id         TEXT NOT NULL
status            TEXT (locked/available/in_progress/complete)
started_at        TIMESTAMPTZ
completed_at      TIMESTAMPTZ
time_spent_mins   INTEGER DEFAULT 0
```

ai_conversations

Tutor chat history

COPY • SQL

```
id              UUID PRIMARY KEY
user_id         UUID (refs auth.users)
context_type    TEXT (lesson/live_session/general)
context_id      TEXT (module_id or session_id)
question        TEXT NOT NULL
response        TEXT NOT NULL
tokens_input    INTEGER
tokens_output   INTEGER
model           TEXT
created_at      TIMESTAMPTZ
```

live_sessions

PDF uploads and analyses

COPY • SQL

```
id                    UUID PRIMARY KEY
user_id               UUID (refs auth.users)
solicitation_number   TEXT
filename              TEXT NOT NULL
file_size_bytes       INTEGER
storage_path          TEXT
analysis_json         JSONB (full Claude analysis)
extracted_text        TEXT (raw PDF text)
analysis_status       TEXT (pending/processing/complete/failed)
error_message         TEXT
tokens_used           INTEGER
analysis_cost_cents   INTEGER
created_at            TIMESTAMPTZ
analyzed_at           TIMESTAMPTZ
```

usage_tracking

Cost/usage events

COPY • SQL

```sql
id              UUID PRIMARY KEY
user_id         UUID (refs auth.users)
event_type      TEXT (tutor_question/pdf_analysis/pdf_upload)
tokens_input    INTEGER
tokens_output   INTEGER
cost_cents      INTEGER
metadata        JSONB
created_at      TIMESTAMPTZ
```

Row Level Security

> **ℹ All tables have RLS enabled**
>
> Admin policies use a helper function public.is_admin() defined as SECURITY DEFINER to avoid infinite recursion in policy checks.

Key policies:

- Users can only SELECT/INSERT/UPDATE their own rows
- Admins can SELECT all rows (not update)
- Auto-triggers create profile + seed 30 modules on signup
- Trial ends 4 days after signup via trigger

Storage Bucket

solicitations bucket (private):

- Users can only upload/view/delete their own files
- Files stored under `{user_id}/{timestamp}-{filename}` path pattern
- 3 policies (INSERT, SELECT, DELETE) enforce user isolation

6. Authentication Flow

Signup

1. User submits signup form at `/signup`
2. Supabase creates row in `auth.users`
3. Trigger `on_auth_user_created` fires → creates `profiles` row with `trial_ends_at = NOW + 4 days`
4. Trigger `on_profile_created` fires → seeds 30 `module_progress` rows (101_1 = available, rest = locked)

5. Supabase sends verification email
6. User clicks link → `/auth/callback` route → redirects to `/dashboard`

Login

7. User submits form at `/login`
8. `signInWithPassword` authenticates
9. Supabase sets httpOnly cookies
10. `proxy.ts` middleware validates on every request
11. User redirected to `/dashboard`

Protected Routes

proxy.ts runs on every request, checks user session:

- Authenticated users on `/login` or `/signup` → redirect to `/dashboard`
- Unauthenticated users on protected routes → redirect to `/login`
- Public routes: `/`, `/login`, `/signup`, `/auth/callback`

7. Curriculum System

Structure

30 modules across 4 levels defined in **lib/curriculum.ts**:

COPY • STRUCTURE

```
Level 101 Foundation — 5 modules (amber)
  101_1, 101_2, 101_3, 101_4, 101_cap

Level 201 Navigation — 11 modules (sage)
  201_1 through 201_10, 201_cap

Level 301 Analysis — 8 modules (navy)
  301_1 through 301_7, 301_cap

Level 401 Mastery — 7 modules (rust)
  401_1 through 401_6, 401_cap
```

Lesson Content

Stored in **lib/lessons.ts** as HTML strings returned by functions:

COPY • TYPESCRIPT

```
function lesson_101_1(): string {
  return `<div class="lesson-header">...`;
```

```
}

const LESSON_HANDLERS = {
  "101_1": lesson_101_1,
  // ...all 30
};
```

HTML uses custom classes rendered by CSS in **globals.css**:

- `.lesson-header` — hero section
- `.level-tag` — eyebrow
- `.lesson-intro` — italic serif intro
- `.callout.info/success/warning/danger` — colored callouts
- `.card`, `.card-dark`, `.card-accent` — content cards
- `.sol-section` + `.section-label` + `.section-content` — mono code blocks
- `.dollar-amount` — amber monospace numbers

Progress Logic

- Visiting `/lesson/{moduleId}` transitions `available` → `in_progress`
- Clicking "Mark complete" transitions to `complete` and unlocks next module
- "Mark complete" can toggle back to `in_progress` for review
- Reset happens by setting `101_1` to `available` and all others to `locked`

8. AI Tutor System

Lesson Tutor — /api/tutor

Student on a lesson asks question → API route:

- Validates user auth
- Loads lesson HTML content
- Strips HTML tags (4000 chars context)
- Builds system prompt with lesson context
- Calls Claude Sonnet 4.5 with conversation history
- Saves Q+A to `ai_conversations` table
- Tracks tokens and cost in `usage_tracking`
- Returns answer JSON

Analysis Tutor — /api/analysis-tutor

Student on analysis results asks question → API route:

- Validates user auth + session ownership
- Loads full analysis JSON (8000 char context)

- Builds system prompt with current tab context
- Calls Claude Sonnet 4.5
- Saves to database
- Returns answer JSON

Both Tutors Share

- Model: `claude-sonnet-4-5`
- Max tokens: `1024`
- Keep responses under 200 words
- Context-aware per lesson or solicitation

9. PDF Analysis Pipeline

Upload Flow — /api/upload-solicitation

12. **Client uploads PDF** via multipart form data
13. **Server validates** auth, file type (PDF), size (< 10MB)
14. **File uploads to Supabase Storage** at `{user_id}/{timestamp}-{filename}`
15. **Creates live_sessions row** with status `processing`
16. **Extracts text** using `pdf-parse` library (server-side Node.js)
17. **Sends to Claude** with structured analysis prompt from `lib/analysis-engine.ts`
18. **Parses JSON response** into Analysis schema
19. **Updates live_sessions** with `analysis_json` and status `complete`
20. **Returns session ID** to client

Analysis Schema

Defined in **lib/analysis-engine.ts** as TypeScript types. Claude returns JSON matching:

COPY • TYPESCRIPT SCHEMA

```
{
  solicitation_overview: { number, type, issued_by, ... },
  what_is_this: string,
  should_you_bid: { recommendation, reasoning, green_flags, red_flags },
  sections: {
    A: { found, raw_summary, key_values, teaching, alternatives_present, action_items },
    B: { ... },
    // ... all 10 sections
    D: {
      // Section D has expanded packaging codes
      key_values: {
        preservation_material_code, preservation_material_desc,
        cushioning_material_code, unit_container_code,
        pack_code, marking_code, special_requirements_code,
        method_of_preservation, opi_code, cleaning_drying_code,
```

```
        qup, mil_dtl_117_type, shelf_life_code, raw_code_block,
        // ...and more
      }
    }
  },
  procurement_history: [{ cage, contract_number, quantity, unit_price, ... }],
  price_analysis: { lowest, highest, average, most_recent, trend, ... },
  compliance_checklist: [{ item, required, explanation }],
  go_no_go_checklist: [{ question, answer, importance }],
  beginner_summary: { what_they_need, total_effort, ... }
}
```

Results View

14 tabs rendered dynamically:

- **Overview** — hero summary + Should You Bid card
- **Sections A–J** — 10 section tabs with key values, teaching, action items
- **History** — price stats + award history table
- **Checklist** — compliance items with required badges
- **Go/No-Go** — big recommendation + decision checklist

Right sidebar on all tabs: **Analysis Tutor** that knows the full JSON analysis.

Packaging Code Decoder

lib/packaging-codes.ts contains the full MIL-STD-2073-1E database:

- 13 categories (preservation material, cushioning, unit container, etc.)
- 150+ individual codes with educational descriptions
- Each code: short and full description
- Used in Section D tab to decode raw codes into human explanations

10. Design System Details

CSS Custom Properties (Tailwind v4)

All tokens defined in **app/globals.css** using the **@theme** directive:

COPY • CSS

```
@theme {
  /* Foundation */
  --color-bg: #F6F1E8;
  --color-surface: #FBF7EF;
  --color-card: #FFFFFF;
  --color-line: #E6DDCC;
```

```
  /* Ink */
  --color-ink: #1B1A17;
  --color-ink-soft: #4A4742;
  --color-ink-muted: #8A857C;

  /* Primary (amber) */
  --color-primary: #B26B1F;
  --color-primary-soft: #F4E4CC;

  /* Level accents */
  --color-sage: #5A7A5A;      /* Level 201 */
  --color-navy: #1F2D4A;      /* Level 301 */
  --color-rust: #9A3D1A;      /* Level 401 */

  /* Fonts */
  --font-serif: var(--font-newsreader);
  --font-sans: var(--font-inter);
  --font-mono: var(--font-jetbrains);

  /* Spacing scale */
  --spacing-space-1: 4px;   /* ... */  --spacing-space-16: 64px;
}

[data-theme="dark"] {
  /* Dark mode overrides */
}
```

Typography Classes

Applied via component classes in **globals.css**:

Class	Definition
`.display-xl`	54px serif for hero
`.display-l`	46px serif for H1
`.display-m`	28px serif for H2
`.display-s`	19px italic serif for accents
`.body-l`	16.5px body
`.body-m`	14px UI text
`.body-s`	13.5px small text
`.meta`	11px JetBrains mono uppercase

Class	Definition
`.numeric`	40px tabular serif

Layout Tokens

- Sidebar: 260px fixed width
- Top bar: 56px fixed height
- Tutor rail: 360px right sidebar
- Article max-width: 640px (lesson measure)
- Page padding: 40px top, 56px sides
- All surfaces: 0px radius (editorial feel)

11. What's Left To Build

Phase 8 — Stripe Subscriptions

Goal: Enforce 4-day trial, collect payment, handle subscription lifecycle.

Files to Create

COPY • FILE TREE

```
app/(dashboard)/billing/
  ├── page.tsx                      # Subscription status page
  └── pricing/page.tsx               # Plans page

app/api/
  ├── stripe/
  │   ├── checkout/route.ts          # Create checkout session
  │   ├── portal/route.ts            # Customer portal redirect
  │   └── webhook/route.ts           # Stripe events handler

lib/
  └── stripe.ts                      # Stripe client + helpers
```

Implementation Steps

21. Sign up at Stripe.com, get test keys
22. Add env vars: `STRIPE_SECRET_KEY`, `STRIPE_PUBLISHABLE_KEY`, `STRIPE_WEBHOOK_SECRET`
23. Create Stripe Products + Prices in dashboard
24. Install `stripe` and `@stripe/stripe-js` packages (already installed)
25. Build checkout flow using Stripe Checkout

26. Handle webhooks: `customer.subscription.created`, `customer.subscription.updated`, `customer.subscription.deleted`, `invoice.payment_failed`
27. Update `profiles.subscription_status` from webhook events
28. Add middleware check in `proxy.ts` — if `trial_ends_at < NOW()` AND `subscription_status != 'active'`, redirect to `/billing/pricing`
29. Build billing page showing current plan, next billing date, cancel button
30. Customer portal for self-service billing

> ⚠ **Pricing Recommendation**
> Single $49/month tier, unlimited everything, 4-day free trial, no credit card required to start trial.

Phase 9 — Admin Dashboard

Goal: Let admin user manage all users, view analytics, grant manual licenses.

Files to Create

COPY • FILE TREE

```
app/(dashboard)/admin/
  ├── page.tsx                      # Admin home with stats
  ├── users/
  │   ├── page.tsx                  # User list table
  │   └── [userId]/page.tsx         # User detail + manual license
  └── analytics/
      └── page.tsx                  # Usage + revenue charts
```

Implementation Steps

31. Add admin-only middleware guard for `/admin/*` routes
32. Build user table: email, signup date, status, last active, total cost
33. User detail page: progress, subscription, usage history
34. Manual actions: grant/extend license, reset password, toggle admin flag
35. Analytics dashboard: new signups, active users, revenue, AI cost vs revenue

Phase 10 — Polish & Error Handling

Goal: Production-quality UX with proper error states.

Work Items

- **Error boundaries** on all major routes (lesson, analysis, dashboard)
- **Loading states** with skeleton UIs instead of blank pages
- **Toast notifications** using sonner for success/error messages
- **Empty states** (no solicitations uploaded yet, no progress, etc.)

- **Form validation** with proper error messages everywhere
- **Mobile responsiveness** — currently desktop-first, needs breakpoints
- **Dark mode verification** — test every page in dark theme
- **Accessibility** — aria labels, keyboard navigation, focus rings
- **404 and error pages** with helpful copy
- **Rate limiting** on API routes to prevent abuse

Phase 11 — Deployment to Vercel

Goal: Launch production instance at a custom domain.

Steps

1. **Push to GitHub**
   - Create new GitHub repo
   - `git init && git add . && git commit -m "Initial"`
   - `git remote add origin` + push
2. **Supabase Production Setup**
   - Create separate production Supabase project (or use existing if ready)
   - Run all SQL migrations on production
   - Set up production email service (Resend recommended over Supabase defaults)
3. **Stripe Production**
   - Activate Stripe account (requires business verification)
   - Create production Products + Prices
   - Copy live keys to Vercel env vars
4. **Vercel Deployment**
   - Sign in to Vercel with GitHub
   - Import the repo
   - Add environment variables (all from `.env.local` plus Stripe live keys)
   - Deploy
   - Vercel auto-deploys on every push to main
5. **Custom Domain**
   - Purchase domain (if not already owned)
   - Add domain in Vercel
   - Update DNS records
   - Wait for SSL provisioning
6. **Production Checks**
   - Verify Supabase email templates point to production URL
   - Verify Stripe webhooks point to production URL
   - Test full signup → trial → payment → lesson → analysis flow
   - Set up error monitoring (Sentry recommended)

Phase 12 — Final Documentation

Goal: Complete handoff documentation for future developers.

Documents to Create

- **README.md** — Setup instructions for local dev
- **ARCHITECTURE.md** — Detailed technical decisions + patterns
- **DATABASE.md** — Full schema + RLS policies + queries
- **API.md** — All API routes documented
- **DEPLOYMENT.md** — Vercel deployment runbook
- **ADMIN_GUIDE.md** — How to manage users as admin
- **USER_GUIDE.md** — End-user help documentation

12. Known Issues and Technical Debt

Current Limitations

- **No rate limiting on AI endpoints** — users could spam questions. Fix: add rate limit middleware using Redis or database.
- **PDF analysis uses single Claude call** — for very large PDFs (>200 pages), context window could be exceeded. Fix: chunked analysis for large docs.
- **No tests** — zero automated tests. Should add Playwright E2E tests at minimum.
- **Middleware warning** — Next.js 16 renamed `middleware.ts` to `proxy.ts` but still shows deprecation. Resolve when stable.
- **No internationalization** — English only. Fix if expanding internationally.
- **Progress calculation shows 31 modules for original test user** — minor cosmetic issue from initial test data. New users see correct 30.

Code Quality Items

- Some components still have `any` types in a few places
- Error messages could be more user-friendly
- Analysis Tutor doesn't yet save conversation history to database (only lesson tutor does)
- No optimistic UI updates for "Mark complete"
- PDF upload has no progress bar during server processing (only during upload)

13. Critical Implementation Details Future Devs Must Know

Supabase RLS Recursion Trap

> ⚠ **DO NOT query profiles directly in admin policies**
>
> Writing admin policies that query the profiles table directly causes infinite recursion. Always use the public.is_admin() helper function, defined as SECURITY DEFINER to bypass RLS.

COPY • SQL

```sql
CREATE OR REPLACE FUNCTION public.is_admin()
RETURNS BOOLEAN
LANGUAGE sql
SECURITY DEFINER
SET search_path = public
STABLE
AS $$
  SELECT COALESCE(
    (SELECT is_admin FROM public.profiles
     WHERE id = auth.uid()),
    FALSE
  );
$$;
```

Tailwind v4 Differences

This project uses **Tailwind v4**, NOT v3. Key differences:

- No `tailwind.config.ts` file needed
- CSS-first configuration using `@theme` directive in `globals.css`
- `@import "tailwindcss"` instead of `@tailwind` directives
- Dark mode via `data-theme` attribute, configured in CSS
- Custom utilities defined using native CSS in `@layer` blocks

Next.js 16 / App Router

- All components default to Server Components
- Must add `"use client"` directive for client components
- `params` in dynamic routes is now a Promise — must `await params`
- Middleware file renamed to `proxy.ts` (still works as `middleware.ts` but shows warning)
- Use `cookies()` from `next/headers` in Server Components

Supabase SSR Pattern

The current pattern uses **@supabase/ssr** (NOT **@supabase/auth-helpers-nextjs** which is deprecated):

COPY • TYPESCRIPT

```
// Server components
const supabase = await createClient();  // from lib/supabase/server

// Client components
const supabase = createClient();  // from lib/supabase/client

// Middleware
await updateSession(request);  // from lib/supabase/middleware
```

> ⚠ **Auth check pattern**
>
> Always use `await supabase.auth.getUser()` (not `getSession()`) for auth checks.

Claude API Patterns

- Use `claude-sonnet-4-5` model (not `claude-3-5-sonnet-20241022`)
- Max tokens for tutor: `1024`
- Max tokens for analysis: `8000+`
- Analysis prompt must be explicit about JSON structure
- Parse response with fallbacks for truncation/markdown wrapping
- Always save token counts to `usage_tracking` for cost monitoring

14. Starting A New Conversation

What To Tell The New AI Assistant

Paste this summary into the new chat:

COPY • PASTE-READY PROMPT

```
I'm continuing work on Solicitation Academy — a Next.js 16 + TypeScript
+ Supabase SaaS teaching DLA DIBBS contracting. I have completed
Phases 1-7 (foundation, auth, database, dashboard, curriculum with
30 lessons, AI Tutor with Claude, PDF analysis with 14-tab results
view). I need to continue with Phase 8 (Stripe subscriptions with
4-day trial), Phase 9 (admin dashboard), Phase 10 (polish), Phase 11
(Vercel deployment), Phase 12 (documentation). Full handoff doc
attached.
```

What The New AI Should Know

- **Stack is locked in** — don't suggest changing to Next.js 15 or Tailwind v3
- **Design system is fixed** — warm editorial aesthetic, don't suggest modernizing

- **User is non-technical** — give paste-ready code, explain every step
- **Build is production-quality** — no shortcuts, follow existing patterns
- **All code should match existing style** — same component structure, naming, styling
- **Testing is manual** — user runs `npm run dev` and checks browser after each change
- **One phase at a time** — complete each phase fully before starting next
- **Real test data exists** — Supabase project has tables, RLS, test user, test session

Credentials and Environment

User has accounts at:

- **Supabase** — project already configured with schema
- **Vercel** — not yet deployed
- **Anthropic** — API key in `.env.local`
- **Stripe** — needs to sign up / verify account creation

Current Working State

- Dev server: `npm run dev` runs at http://localhost:3000
- User is logged in with test account `tah@tehmhand.com`
- Has completed some lessons (test progress exists)
- Has uploaded one test PDF that completed analysis successfully
- All 30 lessons render correctly
- Both AI Tutors work
- Upload/analysis pipeline works

15. Quick Reference — File Cheat Sheet

When you need to find or modify something, here's where it lives:

What	File
Design tokens (colors, fonts, spacing)	`app/globals.css`
Add a new page	Create folder in `app/(dashboard)/newpage/page.tsx`
Add sidebar navigation item	Edit `components/layout/sidebar.tsx`
Add a new module to curriculum	Edit `lib/curriculum.ts` + `lib/lessons.ts`
Modify AI Tutor prompt	`app/api/tutor/route.ts`
Modify analysis prompt	`lib/analysis-engine.ts`
Add new Supabase table	Supabase SQL Editor + regenerate types

What	File
Regenerate TypeScript types	`npx supabase gen types typescript --project-id X > types/database.types.ts`
Protected route logic	`proxy.ts` and `lib/supabase/middleware.ts`
Theme colors for dark mode	`[data-theme="dark"]` in `app/globals.css`
Button/input/card styles	`components/ui/` folder
Claude API calls	`app/api/tutor/route.ts`, `app/api/analysis-tutor/route.ts`, `app/api/upload-solicitation/route.ts`

16. Launch Checklist

When ready for production, verify every item:

	Item
☐	Phase 8 Stripe subscriptions complete
☐	Phase 9 Admin dashboard complete
☐	Phase 10 Polish complete (error handling, loading states, mobile)
☐	Supabase production project created with full schema
☐	Supabase email templates updated with production URL
☐	Stripe production account activated
☐	Stripe live keys configured in Vercel
☐	GitHub repo created and connected to Vercel
☐	Custom domain purchased
☐	DNS records pointing to Vercel
☐	SSL certificate active
☐	Terms of Service page

	Item
☐	Privacy Policy page
☐	Cookie consent banner (if required by jurisdiction)
☐	Error monitoring (Sentry or similar)
☐	Analytics (Plausible or PostHog)
☐	Test full user journey in production
☐	Backup strategy for Supabase (automated daily)
☐	Support email setup
☐	Welcome email flow tested
☐	Trial expiration email tested

E N D O F H A N D O F F G U I D E

Solicitation Academy SaaS • Version 1.0